SatsPath Implementations

This document maps the SatsPath Protocol v1 specification to the current repository implementation.

The implementation must be understood as a protocol stack, not as a single P2P system. P2P is one transport implementation among several.

Repository Layout

```
crates/satspath-core      protocol data types, signatures, resolvers, validation
crates/satspath-router    quote response contract and route selection
crates/satspath-cli       command-line reference client
crates/satspathd          local daemon and HTTP API
sdk/satspath-p2p          optional Pear/Holepunch transport
docs/                     protocol and operational documentation
```

Core Protocol Types

Implemented in:

`crates/satspath-core/src/profile.rs`

Spec mapping:

Spec object	Rust type
`PaymentProfile`	`PaymentProfile`
`SignedPaymentProfile`	`SignedPaymentProfile`
`PaymentMethod.Lightning`	`PaymentMethod::Lightning`
`PaymentMethod.Onchain`	`PaymentMethod::Onchain`
`PaymentMethod.Ark`	`PaymentMethod::Ark`
Invite	`Invite`, `InviteRecord`

Signature and Safety Validation

Implemented in:

`crates/satspath-core/src/crypto.rs`
`crates/satspath-core/src/validation.rs`

Responsibilities:

- Generate protocol identity keypairs.
- Sign public profiles.
- Verify signed profiles.
- Compute identity fingerprints.
- Reject malformed public keys.
- Reject private material in public protocol objects.
- Validate Lightning addresses, Bitcoin addresses, Ark URLs, and compressed pubkeys.

Protocol identity keys are not wallet spending keys.

Resolver Implementations

Implemented in:

`crates/satspath-core/src/resolver.rs`
`crates/satspath-core/src/resolvers/`

`crates/satspath-core/src/registry.rs`
`crates/satspath-core/src/peer_registry.rs`

Current resolver surfaces:

Resolver	File	Status
Local registry	`registry.rs`	Active
Local peer registry	`peer_registry.rs`	Active local storage
BIP-353	`resolvers/bip353.rs`, `bip353.rs`	Resolver and DNS primitives; strict DNSSEC fails closed without validator
HTTPS	`resolvers/http.rs`	Active
Nostr	`resolvers/nostr.rs`	Active NIP-05 + kind 30078 resolver
Platform	`resolvers/platform.rs`	Scaffold

Resolver chain behavior is implemented by `ChainResolver`.

Quote Response and Routing

Implemented in:

`crates/satspath-router/src/quote_response.rs`
`crates/satspath-router/src/router.rs`
`crates/satspath-router/src/fees.rs`
`crates/satspath-router/src/lightning.rs`

Spec mapping:

Spec behavior	Implementation
Resolve identifier	`quote_inner` + `ProfileResolver`
Verify profile	`verify_signed_profile`
Check expiry	`check_profile_expiry`
Select route	`select_route`, `select_route_with_fees`
Build payment payload	`build_qr_payload`
Stable response	`QuoteResponse`

`QuoteResponse` status values:

`ok`
`not_registered`
`no_route`
`invalid_signature`

These values are the UI and API contract.

Daemon Implementation

Implemented in:

`crates/satspathd/src/main.rs`

The daemon exposes the protocol over a local HTTP API:

Endpoint	Purpose
`GET /health`	Liveness
`GET /v1/status`	Local daemon/protocol status
`GET /v1/node`	Aggregate status, profile, peers, connections
`GET /v1/profile`	Local wallet profile state
`PUT/POST /v1/profile`	Create or update local public profile
`POST /v1/profile/methods`	Update receive methods
`POST /v1/resolve`	Resolve a local profile
`POST /v1/quote`	Protocol quote response
`POST /v1/pay`	Wallet handoff using protocol quote response
`POST /v1/dns/resolve`	BIP-353/DNS resolution
`GET /v1/peers`	Local peer registry view
`GET /v1/connections`	Transport/connection diagnostics

`/v1/pay` does not move funds. It returns a wallet handoff containing a public payment payload and QR SVG.

CLI Implementation

Implemented in:

`crates/satspath-cli/src/`

Important commands:

Command area	Protocol role
`register`	Create signed public profile
`wallet`	Manage local receive profile
`quote`	Produce quote response
`pay`	Preview/handoff flow
`dns`	BIP-353 resolver tooling
`peer export/import`	Manual transport for signed profiles

The CLI is a reference client for local development and protocol testing.

P2P Transport Implementation

Implemented in:

`sdk/satspath-p2p/`

This SDK is an optional transport. It publishes and resolves signed profiles over Pear/Holepunch. It must be treated as a resolver transport, not the whole protocol.

Conformance requirements:

- Return `SignedPaymentProfile` objects.
- Verify before import or route use.
- Never treat peer connectivity as profile ownership.
- Never carry wallet private material.

Wire behavior is documented in wire_p2p.md.

Current Gaps

Known v1 implementation gaps:

- DNSSEC strict mode needs a local DNSSEC-validating resolver to fully trust BIP-353 on mainnet.
- Nostr publishing is not yet exposed as a Rust CLI command; use a Nostr client to publish the kind 30078 event. Platform resolvers are scaffolds.
- Resolver provenance is not yet carried through `ProfileResolver`, so signed-profile quote responses mark `identifier_verified: false` unless a direct BIP-353 preview provides DNSSEC validation.
- Mainnet payment execution is intentionally not implemented.
- P2P wire envelopes should be formalized in the SDK to match wire_p2p.md.
- Method ownership proofs exist in core but need broader resolver integration.

Conformance Checklist

An implementation in this repo should be considered conformant when it:

- Uses `SignedPaymentProfile` for receiver data.
- Verifies signatures before routing.
- Applies expiry checks.
- Rejects unsafe private material.
- Produces the four-status `QuoteResponse`.
- Keeps resolver transports separate from protocol verification.
- Treats Pear/Holepunch as optional transport.
- Returns wallet handoff data instead of executing mainnet payments.